

Rapport Technique de Session

Moteur de Tests Fonctionnels Intelligent

Implémentation Complète — Évaluation UX Avancée

Auteur :	Ilyes Chaabani	Date :	23 Juin 2026
Projet :	PFE — Automatisation Intelligente des Tests	Module :	ms-execution / Évaluation UX

Ce rapport documente l'implémentation d'un moteur de tests fonctionnels intelligent intégré à la plateforme d'automatisation des tests (PFE 2026). Le moteur détecte automatiquement les formulaires d'une application web, dérive des cas de test déterministes, et les évalue via trois oracles indépendants : l'API HTML5 Validity, la détection d'erreurs DOM, et la capture réseau XHR/fetch. Il inclut un mécanisme HITL (Human-in-the-Loop) pour les verdicts ambigus, un mode Pause/Reprendre, et la persistance complète des résultats en base PostgreSQL.

4	3	HITL	5 min
Scénarios de test	Oracles indépendants	Validation humaine	Timeout testeur

Table des Matières

1.	Contexte et Objectifs	
2.	Architecture du Moteur	
3.	Détection de Formulaires	
4.	Cas de Tests Dérivés	
5.	Les Trois Oracles Indépendants	
6.	HITL — Human-in-the-Loop	
7.	Mode Supervisé vs Automatique	
8.	Pause / Reprendre	
9.	Persistance des Résultats	
10.	Streaming Temps Réel	
11.	Docker Android Emulator	
12.	Fichiers Créés / Modifiés	
13.	Résultats de Test en Session	
14.	Ce Qui Reste à Faire	
15.	Prérequis de Déploiement	

1. Contexte et Objectifs

Cette session a porté sur l'enrichissement du module d'évaluation UX de la plateforme d'automatisation intelligente des tests (PFE 2026). L'objectif principal était de transformer l'agent exploratoire existant — qui naviguait et prenait des screenshots — en un véritable **moteur de test fonctionnel** capable d'évaluer automatiquement la qualité de validation des formulaires d'une application web.

Objectifs de la session

- Détecter automatiquement les formulaires d'une application web et extraire leur schéma complet
- Dériver des cas de test déterministes à partir du schéma HTML réel (required, type, pattern, bornes)
- Exécuter ces tests avec trois oracles indépendants : HTML5 Validity API, DOM Error Detection, capture réseau
- Permettre l'intervention humaine (HITL) pour les verdicts ambigus en mode Supervisé
- Mettre en pause/repandre une évaluation en cours depuis le frontend
- Persister les résultats en base PostgreSQL pour comparaison inter-runs
- Streamer chaque verdict en temps réel vers le frontend via WebSocket STOMP

Contexte technique : Le moteur s'appuie sur Selenium WebDriver (Chrome) pour le web et Appium W3C REST pour Android. Il n'utilise pas le CDP (Chrome DevTools Protocol) mais une approche JavaScript purement portable, compatible avec tous les navigateurs.

2. Architecture du Moteur de Tests Fonctionnels

2.1 Vue d'ensemble des composants

```
AgenticEvaluationService (orchestrateur principal)
|
+-- WebFunctionalTester (moteur de test fonctionnel)
|   |-- detectForms() -> FormSchema[]
|   |-- testForm() -> FunctionalTestResult[]
|   |   |-- runEmptySubmit() EMPTY_SUBMIT
|   |   |-- runInvalidFormat() INVALID_FORMAT (par champ)
|   |   |-- runBoundary() BOUNDARY (par champ)
|   |   |-- +-- runHappyPath() HAPPY_PATH
|   |   |-- +-- record() -> HITL si verdict incertain
|   |
|   +-- HumanInputService waitForInput() — 5 min timeout
```

```
+-- EvaluationStreamService WebSocket STOMP – verdicts temps réel  
+-- FunctionalTestResultRepo PostgreSQL – persistence  
+-- AppiumDriverService Android W3C REST (APK)
```

2.2 Pipeline d'exécution complet

- 1 L'agent navigue la page cible et extrait tous les formulaires via `detectForms()`
- 2 Déduplication par signature — formulaires déjà testés dans ce run sont ignorés
- 3 Pour chaque nouveau formulaire, `testForm()` exécute la batterie complète de scénarios
- 4 Chaque scénario injecte du JavaScript, interagit avec le DOM, lit les trois oracles
- 5 Le verdict est streamé en temps réel (event TEST_VERDICT) et persisté en PostgreSQL
- 6 En mode Supervisé, verdicts NEEDS_REVIEW déclenchent une pause HITL
- 7 En fin d'évaluation, un résumé fonctionnel est préfixé au rapport UX global

3. Détection de Formulaires

3.1 Extraction du schéma via JavaScript injecté

Le service injecte un script JavaScript dans le DOM pour extraire le schéma complet de chaque formulaire. Les champs de type *hidden*, *submit*, *button*, *reset*, *image*, *file* sont exclus.

```
document.querySelectorAll('form').forEach(function(form, fi) {  
  var fields = [];  
  
  form.querySelectorAll('input, textarea, select').forEach(function(el) {  
    if (['hidden', 'submit', 'button', 'reset'].includes(el.type)) return;  
  
    fields.push({  
      name: el.name || el.id,  
      label: el.labels?.[0]?.innerText || el.getAttribute('aria-label') || el.placeholder,  
      type: el.type, required: el.required, pattern: el.pattern,  
      minLength: el.minLength, maxLength: el.maxLength,  
      min: el.min, max: el.max,  
      options: el.tagName==='SELECT' ? [...el.options].map(o=>o.value) : []  
    });  
  });  
  
  forms.push({ index: fi, id: form.id, action: form.action,  
    method: form.method.toUpperCase(), hasSubmit: !!submitBtn, fields });  
});
```

3.2 Modèle FormSchema

Champ	Type	Description
index	int	Position du formulaire dans le DOM (0, 1, 2...)
id / name	String	Identifiants HTML du formulaire
action	String	URL cible de soumission
method	String	GET ou POST
hasSubmit	boolean	Présence d'un bouton submit
fields	List	Champs interactifs extraits

3.3 Modèle FormField

Champ	Type	Description
name / label	String	Nom et libellé du champ (aria-label ou placeholder en fallback)
tag / type	String	input/textarea/select et email/tel/url/number/text...
required	boolean	Champ obligatoire (required ou aria-required='true')
pattern	String	Expression régulière de validation HTML5
minLength / maxLength	Integer	Contraintes de longueur de texte
min / max	String	Contraintes de valeur numérique
options	List	Valeurs possibles d'un champ select

3.4 Déduplication par signature

Chaque formulaire produit une signature = **action + "|" + noms des champs joints**. Un formulaire dont la signature a déjà été testée dans le run courant est ignoré — évite la duplication sur les SPAs qui rechargent les formulaires en AJAX sans changer d'URL.

4. Cas de Tests Dérivés

Quatre scénarios sont dérivés automatiquement du schéma du formulaire. Chaque scénario est exécuté uniquement si sa précondition est satisfaite.

EMPTY_SUBMIT	
Soumission vide	
Précondition :	Au moins un champ required
Action :	Vider tous les champs, soumettre
Attendu :	Champs requis rejetés, aucune navigation
PASS	blocked=true OU erreurs DOM détectées, pas de navigation
FAIL CRITICAL	Navigation OU serveur 2xx — validation absente
WARN MEDIUM	Serveur 4xx mais validation client absente
NEEDS_REVIEW	Aucun signal détecté — HITL en mode Supervisé

INVALID_FORMAT	
Format invalide	
Précondition :	Champ avec type=email/url/tel OU attribut pattern
Action :	Remplir tous les champs valides, injecter une valeur invalide pour le champ cible
Attendu :	Champ rejeté (HTML5 validity OU erreur DOM)
PASS	htmlRejected=true OU erreurs DOM, pas de navigation
FAIL HIGH	Navigation OU serveur 2xx — valeur invalide acceptée
WARN MEDIUM	Serveur 4xx — client-side absent mais serveur rattrape
NEEDS_REVIEW	Aucun rejet détecté

BOUNDARY

Violation des bornes

Précondition :	Champ avec minLength, maxLength, min ou max
Action :	Injecter valeur hors limites (trop court/long ou hors plage numérique)
Attendu :	Contrainte de borne rejetée
PASS	htmlRejected=true OU erreurs DOM
FAIL MEDIUM	Navigation OU serveur 2xx
WARN LOW	Serveur 4xx — borne non vérifiée côté client
WARN	Comportement ambigu — à vérifier

HAPPY_PATH

Données valides

Précondition :	Toujours (dernier scénario)
Action :	Remplir tous les champs avec des données réalistes générées par l'IA
Attendu :	Aucune erreur, navigation OU réponse réseau 2xx
PASS	Navigation OU serveur 2xx (résout les formulaires AJAX silencieux)
FAIL CRITICAL	Serveur 5xx — bug côté serveur
FAIL HIGH	Serveur 4xx — serveur rejette données valides
NEEDS_REVIEW	Erreurs DOM sur données valides — règle métier ?
WARN	Aucune navigation, aucun appel réseau

5. Les Trois Oracles Indépendants

L'originalité du moteur est de combiner trois sources d'information indépendantes pour produire un verdict déterministe. Aucun oracle seul ne suffit — leur combinaison couvre la quasi-totalité des applications web modernes.

Oracle 1 — HTML5 Validity API

Extraction de l'objet **input.validity** via JavaScript injecté :

```
var v = el.validity;

return {

  valid: v.valid,

  valueMissing: v.valueMissing, // champ requis vide

  typeMismatch: v.typeMismatch, // type email/url/tel non respecté

  patternMismatch: v.patternMismatch, // regex pattern non respectée

  tooLong: v.tooLong, // dépasse maxLength

  tooShort: v.tooShort, // inférieur minLength

  rangeOverflow: v.rangeOverflow, // dépasse max

  rangeUnderflow: v.rangeUnderflow, // inférieur min

  message: el.validationMessage || ''

};
```

Critère	Évaluation
Force	Déterministe, natif au navigateur, zéro faux positif
Limite	Ne fonctionne que pour les champs HTML5 natifs (pas les composants React/Vue custom)

Oracle 2 — DOM Error Detection

Scan des sélecteurs CSS standard et des frameworks UI populaires :

```
var sel = "[aria-invalid='true'], .error, .invalid, .is-invalid,"
+ " .field-error, .form-error, [role='alert'],"
+ " .text-danger, .Mui-error, .invalid-feedback, .errorMessage";

scope.querySelectorAll(sel).forEach(function(e) {

  if (e.offsetParent === null) return; // element invisible – ignore

  var t = (e.innerText || e.textContent || '').trim();
```

```

if (t) out.push(t.substring(0, 120));

else if (e.getAttribute('aria-invalid')==='true') out.push('[champ marque invalide]');

});

```

Critère	Évaluation
Force	Couvre React/Vue/Angular et toutes les validations custom avec classes CSS
Limite	Dépend des conventions de nommage CSS du projet — faux négatifs possibles

Oracle 3 — Capture réseau XHR/fetch (nouveau — session courante)

Un intercepteur JavaScript est injecté **avant chaque soumission** :

```

window.__netCalls = [];

(function() {

if (window.__netPatched) return;

window.__netPatched = true;

// Monkey-patch XMLHttpRequest

var origXHR = window.XMLHttpRequest;

window.XMLHttpRequest = function() {

var xhr = new origXHR(); var _url = '';

xhr.open = function(m, u) { _url = u; return origXHR.prototype.open.apply(this, arguments); };

xhr.addEventListener('load', function() {

window.__netCalls.push({url: _url, status: this.status, type: 'xhr'});

});

return xhr;

};

// Wrap fetch

var origFetch = window.fetch;

window.fetch = function(input, opts) {

var url = typeof input === 'string' ? input : input.url;

return origFetch.apply(this, arguments).then(function(res) {

window.__netCalls.push({url, status: res.status, type: 'fetch'});

return res;

});

};

});

```

```
});
};
})();
```

Code HTTP	Signification	Verdict induit
2xx	Serveur a accepté	PASS pour HAPPY_PATH / FAIL pour scénarios négatifs
4xx	Serveur a rejeté	WARN si validation client absente / FAIL si données valides
5xx	Erreur interne serveur	FAIL CRITICAL sur HAPPY_PATH — bug serveur
0	Erreur réseau (CORS...)	Evidence enregistrée, pas de verdict automatique

Fusion des trois oracles — Table de décision

HTML5	DOM Errors	Réseau	Verdict final
invalid	—	—	PASS — validation native active
valid	présentes	—	NEEDS_REVIEW — règle métier ?
valid	absentes	2xx	PASS — formulaire AJAX accepté
valid	absentes	4xx	FAIL HIGH — serveur rejette données valides
valid	absentes	5xx	FAIL CRITICAL — bug serveur
valid	absentes	aucun	WARN — soumission silencieuse résiduelle

6. HITL — Human-in-the-Loop

Quand l'oracle automatique ne peut pas trancher avec certitude, le système suspend le test et demande au testeur de valider le verdict. Ce mécanisme assure une qualité maximale sur les cas ambigus (règles métier, validations AJAX non standard, états intermédiaires).

6.1 Conditions d'activation

Condition	Valeur seuil
Status du verdict	NEEDS_REVIEW
Confiance du verdict	< 0.6 (sur une échelle 0.0–1.0)
Mode de revue	SUPERVISED (sélectionné à la création de l'évaluation)

Les deux conditions (status + mode) doivent être vraies simultanément pour déclencher l'escalade.

6.2 Flux de validation humaine

1. askTesterToValidate() envoie un event NEEDS_INPUT via WebSocket STOMP
2. Le testeur voit la question dans la vue live (bulle orange avec input)
3. humanInputService.waitForInput(evaluationId, 5 minutes) bloque le thread
4. Le testeur saisit : OK (correct) / BUG (problème) / IGNORE (faux positif)
5. Le backend reçoit la réponse et met à jour le verdict en conséquence
6. Le champ human_validated = true est persisté dans PostgreSQL
7. Timeout : si pas de réponse en 5 min, verdict gardé NEEDS_REVIEW et on continue

6.3 Interprétation des réponses

Réponse testeur	Status final	Sévérité	Note
OK / correct	PASS	—	confidence = 1.0, validé humain
BUG / fail / ko / probleme	FAIL	HIGH	confirmé bug par testeur humain
IGNORE / faux / skip	WARN	—	marqué faux positif par testeur
(timeout 5 min)	NEEDS_REVIEW	—	verdict inchangé, test continue

7. Mode Supervisé vs Automatique

Le mode de revue est sélectionné à la création de l'évaluation depuis le frontend. Il s'applique à toute la durée du run.

Fonctionnalité	AUTO	SUPERVISED
Verdicts certains (PASS/FAIL direct)	Automatiques	Automatiques
Verdicts incertains (NEEDS_REVIEW)	Gardés tels quels	Escaladés au testeur
Pause/Reprendre	Disponible	Disponible
Champ human_validated	false	true si validé
Durée d'exécution	Plus rapide	Plus lente (attente humaine)
Qualité des verdicts	Bonne (3 oracles)	Maximale
Cas d'usage	CI/CD, runs automatisés	Recette manuelle, validation critique

8. Pause / Reprendre

8.1 Mécanique de pause non-bloquante

Un **ConcurrentHashMap<Long, Boolean> pauseRequested** stocke les IDs d'évaluations pausées. À chaque itération de la boucle principale (web ET APK), **awaitIfPaused(id)** est appelé avant toute action :

```
private final Set pauseRequested = ConcurrentHashMap.newKeySet();

void awaitIfPaused(Long evaluationId) {
    while (pauseRequested.contains(evaluationId)) {
        Thread.sleep(500); // poll toutes les 500ms sans bloquer le pool
    }
}

public void pauseEvaluation(Long id) { pauseRequested.add(id); }
public void resumeEvaluation(Long id) { pauseRequested.remove(id); }
```

8.2 Endpoints REST

Méthode	Endpoint	Action
POST	/api/functional-evaluation/{id}/pause	Ajoute l'ID à pauseRequested
POST	/api/functional-evaluation/{id}/resume	Retire l'ID de pauseRequested

8.3 Frontend

Deux boutons Pause / Reprendre apparaissent dans la vue live uniquement quand **isRunning = true**. Le bouton affiche un spinner pendant la transition. L'état `paused` est géré localement — aucun polling

backend nécessaire.

9. Persistance des Résultats

9.1 Schéma SQL — migration manuelle obligatoire

Important : ms-execution est en ddl-auto=validate. Ces instructions SQL doivent être exécutées sur Test_platform_db AVANT de redémarrer le service.

```
-- Colonne mode de revue sur la table existante

ALTER TABLE functional_evaluations

ADD COLUMN IF NOT EXISTS review_mode VARCHAR(20) DEFAULT 'AUTO';

-- Table des résultats détaillés

CREATE TABLE IF NOT EXISTS functional_test_results (

  id BIGSERIAL PRIMARY KEY,

  evaluation_id BIGINT NOT NULL,

  form_label TEXT,

  scenario VARCHAR(40), -- EMPTY_SUBMIT|INVALID_FORMAT|BOUNDARY|HAPPY_PATH

  target_field TEXT,

  input_data TEXT,

  expected TEXT,

  observed TEXT,

  status VARCHAR(20), -- PASS|FAIL|WARN|NEEDS_REVIEW

  severity VARCHAR(20), -- CRITICAL|HIGH|MEDIUM|LOW|INFO

  confidence DOUBLE PRECISION,

  evidence TEXT, -- validity + domErrors + network calls

  human_validated BOOLEAN DEFAULT FALSE,

  created_at TIMESTAMP

);

CREATE INDEX IF NOT EXISTS idx_functional_test_results_evaluation

ON functional_test_results (evaluation_id);
```

9.2 Cycle de vie des données

Moment	Action
--------	--------

Avant chaque run	deleteByEvaluationId(id) — nettoie les anciens résultats
Après chaque formulaire testé	persistFunctionalResults() — sauvegarde tous les verdicts du formulaire
Consultation	GET /api/functional-evaluation/{id}/functional-results

9.3 Utilité de la persistance

- **Répétabilité** : comparer les résultats entre deux runs de la même évaluation
- **Traçabilité** : audit trail avec human_validated, evidence détaillée, et confidence
- **Reporting** : base pour générer un rapport PDF détaillé des tests fonctionnels
- **Régression** : détecter de nouveaux bugs entre la version N et N+1

10. Streaming Temps Réel — Nouveaux Événements

10.1 Trois nouveaux types d'événements WebSocket STOMP

Type	Couleur frontend	Déclencheur
FORM_DETECTED	Indigo	Formulaire détecté, démarrage de la batterie de tests
TEST_CASE	Cyan	Démarrage d'un scénario de test individuel
TEST_VERDICT	Vert / Rouge / Orange	Verdict rendu (PASS / FAIL / WARN / NEEDS_REVIEW)

10.2 Exemple de TEST_VERDICT (HAPPY_PATH avec Oracle réseau)

■ [HAPPY_PATH] PASS

Attendu : Soumission réussie : navigation ou message de succes, AUCUNE erreur de validation.

Observe : Donnees valides acceptees (serveur 2xx) – reponse AJAX recue sans navigation.

Reseau : Appels reseau: [fetch 200]

10.3 Améliorations visuelles frontend (EvaluationLiveView.tsx)

- **Coloration dynamique** des bulles TEST_VERDICT : vert (PASS), rouge (FAIL), orange (WARN/NEEDS_REVIEW) — basée sur le champ `success` de l'event
- **Bouton 'capture'** sur chaque bulle de verdict : toggle d'une miniature du screenshot pris au moment exact du test — sans quitter la vue
- **Boîte de réponse HITL** avec input + bouton Envoyer, apparaît uniquement en mode SUPERVISÉ sur verdicts incertains
- **Scrolling automatique** vers le dernier message à chaque nouveau verdict

11. Docker Android Emulator (APK)

11.1 Problèmes résolus

Problème	Cause racine	Solution
NoSuchMethodError ClientConfig	Conflit Appium 8.6.0 + Selenium BOM Spring Boot	Suppression totale du java-client, HTTP direct W3C REST (JDK HttpClient)
ANDROID_HOME not set	Appium local (port 4723) recevait les requêtes Docker	Port 4725:4723 pour Docker, 4723 réservé à l'Appium local
Emulateur pas prêt après 180s	APPIUM=true manquant dans la commande docker run	Ajout de -e APPIUM=true, --privileged --device /dev/kvm
Can't find service: settings	Android OS pas encore booté au moment de createSession	waitForDeviceBoot(180) : poll getprop sys.boot_completed == 1
Coordonnées inexactes	IA estime les coords dans l'espace 540px vs 1440px réel	Accessibility tree XML : TAP_ELEMENT par index — coordonnées exactes
"null" dans la vue live	sendActionResult(id, step, true, null) sur succès	Passage du message "Action executée avec succès" à la place

11.2 Architecture finale AppiumDriverService

Le service utilise uniquement **java.net.http.HttpClient** (JDK 11+) pour appeler l'API W3C WebDriver d'Appium — aucune dépendance java-client.

Méthode	Description
ensureEmulatorRunning()	Démarre le conteneur Docker testauto-android avec les flags requis
waitForDeviceBoot(180)	Poll docker exec adb shell getprop sys.boot_completed jusqu'à '1'
waitForAppiumReady(300)	Poll HTTP GET http://127.0.0.1:4725/status jusqu'à ready=true
createSession(apkPath)	POST /session avec capabilities UiAutomator2, capture screenWidth/Height
getInteractiveElements()	Parse le XML accessibility tree, extrait bounds [x1,y1][x2,y2] exacts
tap(x, y)	W3C Actions API — pointerDown + pointerUp avec coordonnées scalées
swipeUp/Down()	W3C Actions API — geste de défilement vertical
typeText(text)	W3C Actions API — séquence de touches clavier

12. Fichiers Créés / Modifiés

12.1 Backend ms-execution (Spring Boot)

Fichier	Statut	Description
WebFunctionalTester.java	NOUVEAU	Moteur complet : 4 scénarios, 3 oracles, HITL, données IA
FunctionalTestResult.java	NOUVEAU	DTO verdict avec Status, Severity, evidence, confiance
FormSchema.java	NOUVEAU	Modèle de schéma de formulaire extrait du DOM
FunctionalTestResultEntity.java	NOUVEAU	Entité JPA pour la table functional_test_results
FunctionalTestResultRepository.java	NOUVEAU	Repository Spring Data JPA
AgenticEvaluationService.java	MODIFIÉ	Intégration moteur, pause/resume, HITL, résumé fonctionnel
UxEvaluationController.java	MODIFIÉ	Endpoints pause/resume/functional-results
UxEvaluationRequest.java	MODIFIÉ	Ajout du champ reviewMode
UxEvaluation.java	MODIFIÉ	Colonne review_mode en base
StreamEvent.java	MODIFIÉ	3 nouveaux types + affichage réseau dans TEST_VERDICT
AppiumDriverService.java	RÉÉCRIT	Zéro java-client, HTTP REST W3C direct
db/functional_testing.sql	NOUVEAU	Script de migration manuelle DDL

12.2 Frontend Next.js

Fichier	Statut	Description
EvaluationLiveView.tsx	MODIFIÉ	Couleurs dynamiques PASS/FAIL/WARN, thumbnail screenshot sur verdict
useEvaluationStream.ts	MODIFIÉ	Ajout FORM_DETECTED, TEST_CASE, TEST_VERDICT aux types
lib/api-client.ts	MODIFIÉ	pauseEval, resumeEval, getFunctionalResults, FunctionalTestResultDto
intelligence/create/page.tsx	MODIFIÉ	Sélecteur reviewMode (Automatique / Supervisé)

intelligence/[id]/page.tsx	MODIFIÉ	Boutons Pause / Reprendre avec gestion d'état
----------------------------	---------	---

13. Résultats de Test en Session

Une capture d'écran partagée en cours de session confirme le fonctionnement end-to-end du moteur HITL en mode Supervisé. Le test a porté sur un formulaire de filtrage Drupal exposé-filters.

Formulaire testé

Propriété	Valeur
Identifiant DOM	views-exposed-form-offres-block-1
Nombre de champs	1 (champ de filtre)
Type de formulaire	Filtre GET — Drupal Exposed Filters
Scénario exécuté	HAPPY_PATH

Résultat observé

Étape	Événement	Résultat
1	FORM_DETECTED streamé	Bulle indigo — formulaire détecté avec 1 champ
2	TEST_CASE HAPPY_PATH lancé	Bulle cyan — scénario démarré
3	Verdict NEEDS_REVIEW émis	Bulle orange — soumission silencieuse
4	NEEDS_INPUT envoyé	Question HITL affichée avec input OK/BUG/IGNORE
5	Attente testeur	Boîte de saisie active en bas de la vue live

Impact de l'Oracle réseau ajouté

Ce formulaire Drupal soumet en GET — une requête GET part avec les paramètres de filtre et le serveur répond 200 sans navigation DOM visible (la page se rafraîchit en AJAX). Avec l'Oracle 3 actif, ce cas sera maintenant automatiquement résolu en **PASS** dès que l'intercepteur XHR capture la réponse 200 — sans intervention humaine.

14. Ce Qui Reste à Faire

Haute priorité

Tâche	Estimation
Exécuter la migration SQL sur Test_platform_db avant redémarrage	5 min
Test end-to-end complet avec l'Oracle réseau actif	30 min
Matching Swagger/OpenAPI — lier l'endpoint du formulaire au schéma API	2–3 h

Priorité moyenne

Tâche	Estimation
Version APK du moteur fonctionnel (state diff + détection visuelle)	4–6 h
Injection de sécurité (XSS, SQLi) comme scénarios supplémentaires	3–4 h
Rapport PDF détaillé des tests fonctionnels (séparé du rapport UX)	2–3 h
Comparaison inter-runs ("3 nouvelles régressions vs run précédent")	3–4 h

Priorité basse

Tâche	Estimation
Playwright pour les vrais tests E2E (remplace la simulation JS)	1 journée
Métriques de couverture de formulaires	2–3 h

15. Prérequis de Déploiement

OBLIGATOIRE avant redémarrage : ms-execution utilise ddl-auto=validate — il refusera de démarrer si les colonnes ou tables suivantes sont absentes de la base Test_platform_db.

Script complet à exécuter (fichier : **db/functional_testing.sql**) :

```
-- 1. Mode de revue (AUTO ou SUPERVISED)

ALTER TABLE functional_evaluations

ADD COLUMN IF NOT EXISTS review_mode VARCHAR(20) DEFAULT 'AUTO';

-- 2. Table des resultats de tests fonctionnels

CREATE TABLE IF NOT EXISTS functional_test_results (
```

```
id BIGSERIAL PRIMARY KEY,

evaluation_id BIGINT NOT NULL,

form_label TEXT,

scenario VARCHAR(40),

target_field TEXT,

input_data TEXT,

expected TEXT,

observed TEXT,

status VARCHAR(20),

severity VARCHAR(20),

confidence DOUBLE PRECISION,

evidence TEXT,

human_validated BOOLEAN DEFAULT FALSE,

created_at TIMESTAMP

);

-- 3. Index pour les requetes par evaluation_id

CREATE INDEX IF NOT EXISTS idx_functional_test_results_evaluation

ON functional_test_results (evaluation_id);
```

Ordre de démarrage des services

Ordre	Service	Port	Commande
1	PostgreSQL	5432	docker-compose up -d (infrastructure/)
2	Eureka	8761	mvn spring-boot:run (EurekaServeur/)
3	Auth	8081	mvn spring-boot:run (AuthenticationMicroservice/)
4	ms_gestion	8082	mvn spring-boot:run (ms_gestion/)
5	ms-execution	8083	mvn spring-boot:run (ms-execution/) — après migration SQL
6	Frontend	3000	npm run dev (Frontend/)